

SDK & INTEGRATION GUIDE

Inaya Custody SDK

Client-Side Cryptographic Sovereignty for Distributed Applications

```
npm install @inaya-network/custody-sdk
```

SECTION 01

Overview & Core Philosophy

Encrypt and fragment multi-tenant data structures in under 5 lines of code.

The Inaya SDK empowers distributed applications (dApps) to achieve absolute client-side data sovereignty. By abstracting complex browser-layer PBKDF2 key derivation, AES-GCM-256 binary midpoint sharding, and automated smart contract gas tracking, developers can encrypt and fragment multi-tenant data structures with minimal integration overhead.

PBKDF2 KEY DERIVATION	AES-GCM-256 ENCRYPTION	DUAL-VECTOR SHARDING
EVM LEDGER ATTESTATION	IPFS SWARM ROUTING	

1.1 Rapid Installation Matrix

To inject the cryptographic custody layer directly into your local Node.js environments or React / Next.js frameworks, execute the standard package delivery routine via your project terminal:

```
$ npm install @inaya-network/custody-sdk
```

Requirements — Node.js 18+ and a Web3 wallet provider (MetaMask or equivalent) available in the target browser runtime for wallet-signed ledger attestation calls.

SECTION 02

Step-by-Step Implementation Workflow

1

Initialize & Derive Local Vault Key Wrappers

Before files traverse the public network, a zero-knowledge master wrapper key must be derived locally within the client browser runtime layer using PBKDF2 parameters.

JAVASCRIPT — VAULT KEY DERIVATION

```
import { InayaKernel } from '@inaya-network/custody-sdk';

// 1. Setup localized parameters inside user browser runtime context
const operationalPasskey = "user_secure_administrative_passkey";
const networkSalt = InayaKernel.generateSecureSalt(16); // 16-byte salt

// 2. Compute the asymmetric key wrapper matrix via 100,000 iterations
const dynamicVaultKey = await InayaKernel.deriveVaultKey({
  passkey: operationalPasskey,
  salt: networkSalt,
  iterations: 100000, // Hardened compliance limit
  algo: "HMAC-SHA256" // Cryptographic lineage tracking standard
});
```

2

Execute Dual-Vector Binary Slicing

Once client-side encryption is achieved, the resulting cipher payload buffer is bisected at its exact mathematical midpoint byte location. Neither shard alone contains contiguous bit structures.

JAVASCRIPT — SLICING & DISPERSAL

```
// 3. Encrypt file using AES-GCM-256 and slice into Alpha & Beta fragments
const targetFileBuffer = fileInput.files[0];
const { shardAlpha, shardBeta } = await InayaKernel.disperseAndSlice({
  file: targetFileBuffer,
  encryptionKey: dynamicVaultKey
});

// Structural outcome: neither shard alone contains contiguous bit structures
console.log(`Shard Alpha Matrix Stream Generated successfully.`);
console.log(`Shard Beta Matrix Stream Generated successfully.`);
```

3

Network Routing & EVM State Attestation Settlement

The generated shards are automatically routed across independent decentralized storage clusters. The final asset tracking metadata index registry pointer is then permanently anchored to the public ledger.

JAVASCRIPT — WALLET CONNECTION & LEDGER ANCHOR

```
// 4. Connect Web3 provider (MetaMask window injection protocol)
const web3Provider = await InayaKernel.connectWallet();

// 5. Trigger the smart contract ledger execution handshake
// Note: this operation deducts the standard baseline rate of 0.1 USDT + 0.1 INAYA per GB
const contractReceipt = await InayaKernel.anchorToLedger({
  provider: web3Provider,
  fileName: targetFileBuffer.name,
  dataShardAlpha: shardAlpha, // Destined for distributed IPFS Swarm A
  dataShardBeta: shardBeta,   // Destined for distributed IPFS Swarm B
  pricingTier: "0.1_USDT_0.1_INAYA" // Dual-asset settlement tier
});

console.log(`State registry transaction successful: ${contractReceipt.transactionHash}`);
```

SECTION 03

Enterprise SLA API Routing (B2B Tiers)

For corporate setups utilizing the Scale Matrix, Established Swarm, or Institutional Node subscription models, the SDK initialization allows hardcoding of premium high-throughput gateway parameters.

Tier	Gateway Behavior
Scale Matrix	Standard telemetry routing, up to 15M requests/month
Established Swarm	Priority routing, up to 150M requests/month
Institutional Node	Dedicated RPC node, unlimited allocation

JAVASCRIPT — ENTERPRISE GATEWAY CONFIGURATION

```
// Configure dedicated premium high-performance RPC endpoints
const enterpriseConfig = {
  apiKey: "INAYA_ENT_SECURE_API_KEY_XXXX",
  gatewayPriority: "ULTRA_HIGH_THROUGHPUT", // Bypasses standard congestion layers
  dedicatedRpcNode: "https://rpc.inaya.network/v1/mainnet/enterprise-node"
};

const premiumInayaClient = new InayaKernel.EnterpriseGateway(enterpriseConfig);
```

SECTION 04

API Quick Reference

Method	Purpose
<code>InayaKernel.generateSecureSalt(bytes)</code>	Generates a cryptographically secure random salt
<code>InayaKernel.deriveVaultKey(params)</code>	Derives the local PBKDF2 vault key wrapper
<code>InayaKernel.disperseAndSlice(params)</code>	Encrypts and bisects the payload into Shard Alpha / Beta
<code>InayaKernel.connectWallet()</code>	Opens the Web3 wallet connection handshake
<code>InayaKernel.anchorToLedger(params)</code>	Submits the asset record to the on-chain registry
<code>InayaKernel.EnterpriseGateway(config)</code>	Instantiates a dedicated enterprise RPC client

Security note — Passkeys and derived vault keys should never be transmitted off-device or logged. Store only the salt and resulting content identifiers (CIDs) in application state.

"Every file, sliced and sovereign — encrypted where it starts, never whole where it travels."

@inaya-network/custody-sdk • v1.0.4-beta • BNB Chain Testnet Architecture